

实验五：用户态虚拟内存管理器（学生版）

1. 实验目标

1. 理解虚拟地址空间、物理驻留内存和页表项之间的关系。
2. 通过一个观察程序同时验证按需分配和 `pagemap` 状态变化。
3. 使用 `SIGSEGV`、`mprotect` 和用户态页表模拟缺页处理。
4. 实现一个带 FIFO 页面置换和 swap 文件的 `mini_vmem` 用户态虚拟内存管理器。

2. 实验环境

- 操作系统：Ubuntu 22.04 LTS 或更高版本。
- 编译器：`gcc`。
- 推荐工具：`htop`。
- 注意：任务一读取 `/proc/self/pagemap` 通常需要 `sudo`；任务二不需要 `root` 权限。

3. 任务一：观察 Linux 的真实虚拟内存行为

补全 `vm_observe.c`。

3.1 为什么做这个任务

在实现自己的虚拟内存管理器之前，先观察 Linux 内核真实做了什么。`mmap` 成功返回时，进程获得的是一段虚拟地址空间；页面是否真正进入物理内存，要等到访问时才能确定。`/proc/self/pagemap` 则能让你看到某个虚拟页在访问前后的页表项变化。

这个任务的目的，是把三个概念连起来：

- `VIRT`：进程拥有的虚拟地址空间；
- `RES/RSS`：实际驻留在物理内存中的页面；
- `pagemap`：虚拟页对应的页表项状态。

3.2 具体要做什么

你需要补全 `vm_observe.c`，让程序完成以下流程：

1. 在 `mmap` 前先暂停，根据程序打印的 PID 启动 `sudo htop -p <PID>`，记录进程初始 `VIRT` 与 `RES/RSS`；
2. 使用 `mmap` 申请一段较大的匿名虚拟内存；
3. 在尚未访问内存时读取首个页面的 `pagemap` 条目，并再次暂停观察 `VIRT` 与 `RES/RSS`；
4. 写入首个页面，再次读取同一页面的 `pagemap` 条目；
5. 逐页写入整段内存，观察 `RES/RSS` 如何变化；
6. 释放映射区域。

3.3 期望得到什么

你能观察到：

- `mmap` 前后对比，`VIRT` 增加明显，但 `RES/RSS` 不会立刻等量增加；
- 首次写入页面后，该页面的 `pagemap` 状态发生变化；
- 逐页写入后，`RES/RSS` 随着访问逐步增长。

报告中提交 `vm_observe.c` 完整代码和运行结果，并说明这些现象如何共同证明“虚拟内存按需映射”。

3.4 待补全代码 `vm_observe.c`

```
#define _GNU_SOURCE

#include <fcntl.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <unistd.h>

#define MB (1024UL * 1024)
#define OBSERVE_SIZE (512UL * MB)
#define PROGRESS_STEP (64UL * MB)
#define PROGRESS_DELAY_SEC 1

static void wait_for_input(void) {
    char line[32];
    if (fgets(line, sizeof(line), stdin) == NULL) {
        exit(0);
    }
}

// 读取并打印 addr 所在虚拟页的 pagemap 条目。
// 这个函数用于观察“某个虚拟页在访问前后，页表项状态是否发生变化”。
// 输出中的 raw 是 64 位原始条目，present 表示该页是否驻留，pfn 表示物理页帧号。
static void check_pagemap(void *addr) {
    uint64_t value = 0;
    size_t page_size = (size_t)sysconf(_SC_PAGESIZE);

    // /proc/self/pagemap 是当前进程的页表观察接口。
    // 读取 PFN 通常需要 sudo 权限。
    int fd = open("/proc/self/pagemap", O_RDONLY);
    if (fd < 0) {
        perror("open pagemap failed");
        exit(1);
    }

    uint64_t vpn;
    // TODO 1: 根据虚拟地址 addr 和 page_size 计算虚拟页号 vpn。

    off_t offset;
```

```

// TODO 2: 计算这个 VPN 对应的 pagemap 文件读取位置。
// 注意: 这里的 offset 是 /proc/self/pagemap 文件内的偏移,
// 不是虚拟地址在页面内部的页内偏移。
// pagemap 中每个虚拟页对应一个 8 字节条目。

// lseek 将文件读写位置移动到 offset。
// 这里的目的, 是跳到目标虚拟页对应的 pagemap 条目开头。
if (lseek(fd, offset, SEEK_SET) == (off_t)-1) {
    perror("lseek failed");
    close(fd);
    exit(1);
}

// read 从当前文件位置读取 8 字节, 也就是一个 pagemap 条目。
// 读取到的 64 位原始值会保存在 value 中, 后面再用位运算解析。
// 63      62      61 ... 55      54 ... 0
//+-----+-----+-----+-----+
//|present|swapped| other flags |      PFN      |
//+-----+-----+-----+-----+
if (read(fd, &value, sizeof(value)) != sizeof(value)) {
    perror("read failed");
    close(fd);
    exit(1);
}

close(fd);

int present;
// Present 位表示这个虚拟页当前是否已经映射到物理内存。
// 在 pagemap 的 64 位条目中, bit 63 是 Present 位。
// TODO 3: 解析 bit 63 的 Present 位。

uint64_t pfn;
// TODO 4: 解析低 55 位 PFN。

// 对比写入前后的输出, 观察 present 和 pfn 是否变化。
printf("addr=%p raw=0x%016lx present=%d pfn=0x%lx\n",
        addr, value, present, pfn);
}

int main(void) {
    size_t size = OBSERVE_SIZE;
    size_t page_size = (size_t)sysconf(_SC_PAGESIZE);

    printf("vm_observe pid = %d\n", getpid());
    printf("page size = %zu bytes = %zu KB\n", page_size, page_size / 1024);
    printf("\n=== 阶段 0: mmap 前 ===\n");
    printf("[现在观察] 用 sudo htop -p <上方 PID> 记录当前 VIRT 与 RES/RSS. \n");
    printf("[下一步] 按回车后, 程序将调用 mmap 申请 %zu MB 虚拟内存. \n", size / MB);
    printf("[预测] VIRT 是否会明显增加? RES/RSS 是否会同步增加? \n");
    wait_for_input();

    printf("准备申请 %zu MB 虚拟内存...\n", size / MB);

```

```

char *ptr;
// TODO 5: 使用 mmap 申请虚拟内存，并赋值给 ptr。
// 权限 prot 应设置为可读写；映射 flags 应设置为私有匿名映射。

if (ptr == MAP_FAILED) {
    perror("mmap failed");
    return 1;
}

printf("mmap 成功，起始地址: %p\n", ptr);

printf("\n=== 阶段 1: mmap 后，但尚未访问页面 ===\n");
check_pagemap(ptr);
// 此时只建立了虚拟地址范围，首个页面还没有被写入。
// 和阶段 0 对比，观察 VIRT 是否增加，以及 RES/RSS 是否只是小幅变化。
printf("[现在观察] 记录当前 VIRT、RES/RSS 和上方 pagemap 输出。\\n");
printf("[下一步] 按回车后，程序将只写入 ptr[0] 所在的首个页面。\\n");
printf("[预测] pagemap 的 present 是否会变化？RES/RSS 是否只小幅变化？\\n");
wait_for_input();

printf("\n=== 阶段 2: 写入首个页面 ===\n");
// TODO 6: 向 ptr[0] 写入任意一个 char 值，触发首个页面实际映射。
// 这里关注的是“发生写访问”这一动作，写入的具体字符不影响观察结论。

check_pagemap(ptr);
printf("[现在观察] 记录写入首个页面后的 pagemap、VIRT 和 RES/RSS。\\n");
printf("[下一步] 按回车后，程序将按 page_size 步长逐页写完整段内存。\\n");
printf("[预测] RES/RSS 会随着页面被写入而逐步增长。\\n");
wait_for_input();

printf("\n=== 阶段 3: 逐页写入整段内存 ===\n");
printf("每写入 %zu MB 暂停 %d 秒，方便在 htop 中观察 RES/RSS 的增长。\\n",
       PROGRESS_STEP / MB, PROGRESS_DELAY_SEC);
for (size_t i = 0; i < size; i += page_size) {
    // TODO 7: 向 ptr[i] 写入任意一个 char 值，触发当前页面实际映射。
    // 循环每次跨过一个 page_size，因此每次写入都会访问一个新的虚拟页。

    size_t visited = i + page_size;
    if (visited % PROGRESS_STEP == 0 || visited == size) {
        printf("已访问: %zu MB / %zu MB\\n", visited / MB, size / MB);
        sleep(PROGRESS_DELAY_SEC);
    }
}

printf("[现在观察] 逐页访问完成，请记录释放前的 VIRT 与 RES/RSS。\\n");
printf("[下一步] 按回车后，程序将释放 mmap 区域；重点观察 VIRT/RES 是否下降。\\n");
wait_for_input();

// TODO 8: 使用 munmap 释放从 ptr 开始、长度为 size 的 mmap 区域。
// munmap 后这段虚拟地址范围不再属于当前进程，不能继续访问。

printf("[现在观察] mmap 区域已释放。请记录释放后的 VIRT 与 RES/RSS。\\n");

```

```
printf("[结束] 按回车后退出程序.\n");
wait_for_input();

return 0;
}
```

编译运行：

```
gcc -Wall -Wextra -std=c11 vm_observe.c -o vm_observe
sudo ./vm_observe
```

运行后按以下步骤操作：

1. 程序先打印 pid，并停在“阶段 0：mmap 前”。此时保持程序暂停。
2. 记下程序打印的 PID。新开一个终端，用该 PID 启动 `htop`，例如程序打印 `vm_observe pid = 13995`，就运行：

```
sudo htop -p 13995
```

记录这个进程的初始 `VIRT` 和 `RES`。

3. 阅读程序中的“下一步”和“预测”提示：按回车后，程序将调用 `mmap` 申请 512MB 虚拟内存。先预测 `VIRT` 和 `RES` 哪个会明显变化，再继续。
4. 程序停在“阶段 1：mmap 后，但尚未访问页面”。记录 `VIRT`、`RES` 和 `pagemap` 输出，并和阶段 0 对比。
5. 阅读程序中的“下一步”和“预测”提示：按回车后，程序将只写入 `ptr[0]` 所在的首个页面。先预测 `pagemap` 和 `RES` 会如何变化，再继续。
6. 程序停在“阶段 2：写入首个页面”之后。记录写入后的 `VIRT`、`RES` 和 `pagemap` 输出，对比写入前后的 `present`、`pfn` 和 `raw value`。
7. 阅读程序中的“下一步”和“预测”提示：按回车后，程序将按 `page_size` 步长逐页写完整段内存。此阶段每写入 64MB 会暂停 1 秒，先预测 `RES` 会如何变化，再继续。
8. 观察阶段 3 的进度输出和 `htop` 中的 `RES/RSS` 变化。程序逐页写入完成后会再次暂停，此时记录释放前的 `VIRT` 和 `RES`，重点观察 `RES` 是否明显增长。
9. 回到程序终端按回车，程序会释放 `mmap` 区域并再次暂停。记录释放后的 `VIRT` 和 `RES`，和释放前对比。
10. 回到程序终端按回车退出。

报告中至少应记录五类信息：`mmap` 前的 `VIRT/RES`，`mmap` 后但访问前的 `VIRT/RES` 和 `pagemap` 输出，写入首个页面后的 `VIRT/RES` 和 `pagemap` 输出，逐页写入后的释放前 `VIRT/RES`，释放 `mmap` 后的 `VIRT/RES`。

4. 任务二：实现 mini_vmem 用户态虚拟内存管理器

补全 `mini_vmem.c`。程序申请 8 个虚拟页，但最多只允许 4 个页面驻留在内存中。访问未驻留页面时触发 `SIGSEGV`，handler 负责装入页面；驻留页已满时使用 FIFO 淘汰页面，并把页面内容写入 `mini_vmem.swap`。这里的 `mini_vmem.swap` 是用普通文件模拟的交换区：它临时保存被换出页面的内容，等页面再次被访问时再读回来。

4.1 为什么做这个任务

任务一观察的是 Linux 内核已经实现好的机制。任务二要求你反过来扮演一个简化版操作系统：自己维护页表状态，自己决定页面什么时候装入，自己选择淘汰哪一页，自己把被淘汰页面保存到 swap 文件，再在后续访问时读回来。

它要帮助你理解：缺页处理不是一个抽象名词，而是一条具体控制流：发生缺页异常、定位地址、查页表、分配或读回页面、更新权限、返回后重新执行指令。

4.2 具体要做什么

程序会先申请 8 个连续虚拟页，并把它们全部设置为 `PROT_NONE`。这意味着任何读写都会先触发 `SIGSEGV`。你需要补全的 `mini_vmem` 需要完成：

1. 根据 `info->si_addr` 判断故障地址属于哪个虚拟页；
2. 如果页面从未装入，就把它清零并设为可读写；
3. 如果页面曾经被淘汰到 swap 文件，就从 swap 文件读回；
4. 如果当前驻留页数量已经达到 4 页，就按 FIFO 选择一个页面淘汰；
5. 淘汰页面时，把页面内容写入它在 swap 文件中的固定位置，并把权限改回 `PROT_NONE`；
6. 更新用户态页表中的状态、dirty 标记和装入时间。

4.3 期望得到什么

程序运行时应能看到清晰的状态变化：

- 第一次访问某个页面时输出 `[fault]` 和 `[load]`；
- 访问超过 4 个不同页面后开始输出 `[evict]`；
- 被淘汰页面再次访问时，会先 `fault`，再从 swap 文件读回；
- 最后重新访问 page 0 时，应能读回最初写入的字符 `A`。

如果最后 page 0 的值不是 `A`，通常说明 swap-out、swap-in 或页表状态更新存在问题。

4.4 待补全代码 `mini_vmem.c`

```
#define _GNU_SOURCE

#include <errno.h>
#include <fcntl.h>
#include <signal.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <unistd.h>
#include <limits.h>

#define NUM_VPAGES 8
#define MAX_RESIDENT 4
```

```

typedef enum {
    // 页面从未被装入过，swap 文件中也没有它的有效内容。
    PAGE_EMPTY = 0,
    // 页面当前驻留在内存中，访问权限应为 PROT_READ | PROT_WRITE。
    PAGE_RESIDENT,
    // 页面曾经驻留过，但现在已被换出到 swap 文件。
    PAGE_SWAPPED
} page_state_t;

typedef struct {
    // 当前虚拟页的状态：空、驻留或已换出。
    page_state_t state;
    // 基础版本采用保守策略：页面装入并允许写后，就把它视为 dirty。
    // dirty 页淘汰时要写 swap。
    int dirty;
    // 页面装入内存时的逻辑时间；FIFO 用它找到最早装入的页面。
    unsigned long load_time;
} page_entry_t;

// 页大小由系统决定，代码不能写死为 4096。
static size_t page_size;
// mini_vmem 管理的连续虚拟内存区间起始地址。
static char *vm_base;
// 用户态页表：第 i 项描述第 i 个虚拟页的状态。
static page_entry_t page_table[NUM_VPAGES];
// 当前驻留页数量，不能超过 MAX_RESIDENT。
static int resident_count;
// 每装入一页递增一次，用来给 load_time 打时间戳。
static unsigned long clock_tick;
// swap 文件描述符：被淘汰的页会写入 mini_vmem.swap。
static int swap_fd = -1;

// 函数实现按 main 的执行路径从上到下排列；这里先给出声明，方便前面的函数调用后面的函数。
static void init_vmem(void);
static void destroy_vmem(void);
static void install_handler(void);
static void segv_handler(int sig, siginfo_t *info, void *context);
static void load_page(int page_no);
static void evict_page(int victim);
static int choose_victim_fifo(void);
static int page_of_addr(void *addr);
static void *page_start(int page_no);
static void full_write(int fd, const void *buf, size_t nbyte);
static void full_read(int fd, void *buf, size_t nbyte);

int main(void) {
    init_vmem();

    printf("mini_vmem start: %d virtual pages, only %d resident pages allowed\n",
        NUM_VPAGES, MAX_RESIDENT);

    for (int i = 0; i < NUM_VPAGES; i++) {

```

```

    // 每次写 p[0] 都可能触发缺页。
    // 前 4 个不同页面会被装入；继续访问新页面时会触发 FIFO 淘汰。
    char *p = vm_base + (size_t)i * page_size;
    printf("\nwrite page %d = %c\n", i, (char)('A' + i));
    p[0] = (char)('A' + i);
}

printf("\nrevisit page 0, should trigger swap-in if it was evicted\n");
// 如果第 0 页之前被换出，这次读取会触发缺页并从 swap 文件恢复内容。
printf("page 0 value = %c\n", vm_base[0]);

destroy_vmem();
return 0;
}

// 初始化 mini_vmem:
// 1. 获取页大小;
// 2. 申请 NUM_VPAGES 个 PROT_NONE 虚拟页;
// 3. 创建 swap 文件;
// 4. 注册 SIGSEGV handler.
static void init_vmem(void) {
    // 获取系统页大小。
    long ps = sysconf(_SC_PAGESIZE);
    if (ps <= 0) {
        perror("sysconf(_SC_PAGESIZE)");
        exit(1);
    }
    page_size = (size_t)ps;

    // 先申请一段完全不可访问的虚拟内存。
    // 后续每次访问都会触发 SIGSEGV，交给 segv_handler 按需装入页面。
    vm_base = mmap(NULL, page_size * NUM_VPAGES, PROT_NONE,
                   MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    if (vm_base == MAP_FAILED) {
        perror("mmap vm");
        exit(1);
    }

    // swap 区用于临时保存“当前不驻留在内存中”的页面内容。
    // 真实系统可能使用磁盘分区或专门的 swap 文件；本实验用普通文件模拟。
    // 创建一个普通文件来模拟 swap 区。
    // O_CREAT: 文件不存在就创建；O_RDWR: 后面既要写出页面，也要读回页面；
    // O_TRUNC: 每次运行实验都从空文件开始，避免读到上一次运行留下的数据；
    // O600: 只有当前用户可读写这个临时 swap 文件。
    swap_fd = open("mini_vmem.swap", O_CREAT | O_RDWR | O_TRUNC, 0600);
    if (swap_fd == -1) {
        perror("open swap");
        exit(1);
    }

    // 把 swap 文件扩展到“能放下 NUM_VPAGES 个页面”的大小。
    // 可以把这个文件想成一排固定大小的格子，每个格子大小都是 page_size:

```

```

// 第 0 页使用第 0 个格子，第 1 页使用第 1 个格子，以此类推。
// 因此第 page_no 页在文件中的起始位置就是 page_no * page_size。
// ftruncate 只是在文件里划出这些格子，不代表页面内容已经写进去了。
if (ftruncate(swap_fd, (off_t)(page_size * NUM_VPAGES)) == -1) {
    perror("ftruncate swap");
    exit(1);
}

// 注册 SIGSEGV 处理函数。
// 之后访问 PROT_NONE 页面时，程序不会立刻结束，而是进入 segv_handler。
install_handler();
}

// 注册 SIGSEGV handler。
// SA_SIGINFO 必须设置，否则 handler 拿不到 info->si_addr。
static void install_handler(void) {
    // sigaction 用来描述“收到某个信号时，应该怎么处理”。
    struct sigaction sa;
    // 先把结构体清零，避免未初始化字段影响信号处理行为。
    memset(&sa, 0, sizeof(sa));
    // 使用三参数形式的 handler: handler(int, siginfo_t *, void *)。
    // 这样 handler 才能通过 siginfo_t 拿到导致缺页的地址。
    sa.sa_flags = SA_SIGINFO;
    // 指定 SIGSEGV 发生时要调用 segv_handler。
    sa.sa_sigaction = segv_handler;
    // handler 运行期间不额外屏蔽其他信号。
    sigemptyset(&sa.sa_mask);

    // 把上面的处理规则安装到 SIGSEGV 上。
    // 之后访问 PROT_NONE 页面时，Linux 会调用 segv_handler，而不是直接结束进程。
    if (sigaction(SIGSEGV, &sa, NULL) == -1) {
        perror("sigaction");
        exit(1);
    }
}

// SIGSEGV 处理函数，相当于 mini_vmem 的“用户态缺页处理入口”。
// 只有访问本实验管理的 PROT_NONE 页面时，才应该在这里完成换页和权限恢复。
// info->si_addr 是触发 fault 的精确虚拟地址，用它可以定位对应的虚拟页号。
static void segv_handler(int sig, siginfo_t *info, void *context) {
    (void)sig;
    (void)context;

    // 根据 fault 地址计算它属于 mini_vmem 中的第几个虚拟页。
    int page_no = page_of_addr(info->si_addr);
    if (page_no < 0) {
        // 地址不属于 mini_vmem 管理区，说明这是普通非法访问，
        // 而不是本实验预期的缺页。恢复默认处理后重新触发 SIGSEGV，
        // 让程序按真实错误崩溃，避免把越界访问误当成缺页处理。
        fprintf(stderr, "invalid access at %p\n", info->si_addr);
        signal(SIGSEGV, SIG_DFL);
    }
}

```

```

        raise(SIGSEGV);
    }

    printf("[fault] addr=%p page=%d\n", info->si_addr, page_no);
    // load_page 负责把目标页调入物理页框，并把该页权限改回可访问。
    // handler 返回后，CPU 会自动重试刚才失败的那条访存指令。
    load_page(page_no);
}

// 将一个故障地址转换成 mini_vmem 管理的虚拟页号。
// 如果地址不属于 vm_base 管理的区域，说明这不是本实验应该处理的缺页。
static int page_of_addr(void *addr) {
    // TODO 1: 判断 addr 是否落在 vm_base 管理的虚拟内存区域内。
    // 可以把 vm_base 看成起点，把 vm_base + NUM_VPAGES * page_size 看成终点。
    // 只有落在 [起点, 终点) 内的地址，才属于 mini_vmem 管理。
    // 合法地址到 vm_base 的距离，就是它在这段虚拟内存中的偏移量。
    // 合法则返回虚拟页号；非法则返回 -1。
    return -1;
}

// 将 page_no 对应的虚拟页装入内存。
// 如果驻留页数量已达上限，需要先淘汰一页。
// 如果该页从未装入，装入时清零；如果该页被换出过，则从 swap 文件读回。
static void load_page(int page_no) {
    void *addr = page_start(page_no);
    // page_no 对应的 swap 槽位固定，swap-in 和 swap-out 使用同一个 offset。
    off_t offset = (off_t)page_no * (off_t)page_size;

    // 内存中最多只允许 MAX_RESIDENT 个页面驻留。
    // 如果已经满了，本次装入前必须先按 FIFO 选择一个驻留页作为 victim。
    if (resident_count >= MAX_RESIDENT) {
        int victim = choose_victim_fifo();
        if (victim < 0) {
            // resident_count 显示已满时，正常应至少找到一个可淘汰的驻留页。
            // 找不到 victim 通常说明页表状态或 resident_count 维护有误。
            fprintf(stderr, "no page can be evicted\n");
            exit(1);
        }
        // 调用 evict_page 完成换出流程，为新页面腾出驻留名额。
        evict_page(victim);
    }

    // TODO 2: 将目标页权限改为 PROT_READ | PROT_WRITE。
    // 只有权限恢复后，才能对该页执行 memset 或从 swap 文件读入数据。

    if (page_table[page_no].state == PAGE_SWAPPED) {
        // TODO 3: 从 swap 文件固定偏移读回该页内容。
        // 这是 swap-in: 页面内容来自它上次被淘汰时写入的 swap 槽位。
        printf("[load] page %d <- swap offset %ld\n", page_no, (long)offset);
    } else {
        // TODO 4: 第一次装入页面时清零。
        // PAGE_EMPTY 页面没有历史内容，清零模拟第一次分配到干净物理页。

```

```

    printf("[load] page %d <- zero page\n", page_no);
}

// TODO 5: 更新目标页表项的state,dirty,load_time和 resident_count。

}

// 根据虚拟页号反推出该页的页起始地址。
// mprotect、swap 读写和清零都应应以页为单位进行。
static void *page_start(int page_no) {
    // TODO 6: 计算 page_no 对应页的起始地址。
    // 第 0 页从 vm_base 开始; 第 1 页从 vm_base + page_size 开始;
    // 第 page_no 页也按同样规律向后移动。
    return NULL;
}

// FIFO 页面置换: 选择最早装入内存的驻留页作为 victim。
// 注意只能在 PAGE_RESIDENT 页面中选择, 不能淘汰空页或已经换出的页。
static int choose_victim_fifo(void) {
    // TODO 7: 在 PAGE_RESIDENT 页面中选择 load_time 最小的一页。
    // load_time 越小, 表示该页越早被 load_page 装入内存。
    // 如果没有驻留页可选, 应返回 -1。
    return -1;
}

// 将一个驻留页换出到 swap 文件。
// victim 是要淘汰的虚拟页号; 换出后该页应重新变成不可访问状态。
static void evict_page(int victim) {
    void *addr = page_start(victim);
    // 每个虚拟页在 swap 文件中有固定槽位:
    // 第 0 页写到 offset 0, 第 1 页写到 offset page_size, 以此类推。
    off_t offset = (off_t)victim * (off_t)page_size;

    // TODO 8: 如果 victim 是脏页, 将整页内容写入 swap 文件的固定偏移。
    // 写入的数据来自当前仍可访问的 victim 页, 长度应为完整一页。

    // TODO 9: 将 victim 页权限改回 PROT_NONE。
    // 这样以后再次访问该页时, 会重新触发 SIGSEGV, 由 handler 负责 swap-in。

    // TODO 10: 更新 victim 页表项和 resident_count。
    // 需要维护 state、dirty 和 resident_count, 使页表状态与换出后的页面位置一致。

    printf("[evict] page %d -> swap offset %ld\n", victim, (long)offset);
}

// 确保完整写出 nbyte 字节。
// 普通 write 可能只写出部分数据, 因此这里用循环保证整页写入 swap 文件。
static void full_write(int fd, const void *buf, size_t nbyte) {
    const char *p = buf;
    size_t done = 0;
    while (done < nbyte) {
        ssize_t n = write(fd, p + done, nbyte - done);
    }
}

```

```

        if (n == -1) {
            if (errno == EINTR) continue;
            perror("write swap");
            exit(1);
        }
        done += (size_t)n;
    }
}

// 确保完整读入 nbyte 字节。
// swap-in 时必须读回整页，否则页面内容会不完整。
static void full_read(int fd, void *buf, size_t nbyte) {
    char *p = buf;
    size_t done = 0;
    while (done < nbyte) {
        ssize_t n = read(fd, p + done, nbyte - done);
        if (n == -1) {
            if (errno == EINTR) continue;
            perror("read swap");
            exit(1);
        }
        if (n == 0) {
            fprintf(stderr, "unexpected EOF while reading swap\n");
            exit(1);
        }
        done += (size_t)n;
    }
}

static void destroy_vmem(void) {
    // 释放虚拟内存管理区，关闭并删除本次实验创建的 swap 文件。
    munmap(vm_base, page_size * NUM_VPAGES);
    if (swap_fd != -1) close(swap_fd);
    unlink("mini_vmem.swap");
}

```

编译运行：

```

gcc -Wall -Wextra -std=c11 mini_vmem.c -o mini_vmem
./mini_vmem

```

5. 实验思考

1. 任务一中，`mmap` 后 `VIRT` 通常会明显增加，而 `RES/RSS` 不会同步增加；写入首个页面后，`RES/RSS` 也可能几乎看不出变化。请结合按需分配、页大小和 `htop` 统计粒度解释这些现象，并说明为什么逐页写完整段内存后现象会更明显。
2. 在任务二中，`segv_handler()` 必须先判断 `info->si_addr` 是否属于 `mini_vmem` 管理区。如果省略这个判

断，直接把所有 `SIGSEGV` 都当作实验缺页处理，可能掩盖哪些真实程序错误？请举出至少两类访问错误并分析后果。

3. 本实验把每个虚拟页固定映射到 swap 文件中的一个槽位。请分析这种设计相比“动态分配空闲 swap 槽位”有什么优点和局限；如果要改成动态槽位分配，页表项至少还需要增加哪些信息？
4. 如果 `evict_page()` 在写入 swap 前就先把页面改成 `PROT_NONE`，再尝试从该页地址读取内容写盘，会发生什么？为什么？
5. 如果一个页面只被读过、从未被写过，基础版本仍可能把它当作 dirty 页面写回。这个保守策略会带来什么额外开销？它为什么仍然能保证正确性？

6. 自选进阶任务（二选一）

进阶任务一：Second Chance 页面置换

在 FIFO 的基础上实现 Second Chance 页面置换。基础版程序直接用指针访问页面，驻留页再次被访问时不会进入 `SIGSEGV` handler，因此进阶版需要增加统一的页面访问接口，用它记录页面是否被访问过。要求：

- 为每个页面维护 `referenced` 位；
- 增加 `vmem_read` / `vmem_write` 这类访问接口，并在接口中维护 `referenced`；
- 修改测试代码，通过访问接口读写页面，而不是直接操作 `vm_base` 指针；
- 选择 victim 时，若页面 `referenced = 1`，先清零并跳过；若 `referenced = 0`，才淘汰；
- 构造访问序列，对比 FIFO 与 Second Chance 的淘汰差异。

进阶任务二：写保护 Dirty Tracking

实现更精确的 dirty tracking。基础版为了保证正确性，会把装入后允许写入的页面都保守视为 dirty；进阶版需要利用页权限，把“首次写入”单独识别出来。要求：

- 页面从 swap 或空页装入并准备好内容后，先设置为只读；
- 第一次写入该页时再次触发 `SIGSEGV`；
- handler 识别这是写保护缺页，将页面改为可读写，并设置 dirty 位；
- 淘汰 clean page 时不写 swap，淘汰 dirty page 时才写 swap；
- 构造读多写少的访问序列，证明 clean page 不会被无意义写回。

7. 实验提交

完成实验报告的填写，以“学号-姓名-操作系统实验五”命名。

将实验报告提交到作业网站：<https://acm.scu.edu.cn/teach/>